Advanced Machine Learning

Emilie Chouzenoux[(1)], Frédéric Pascal[(2)], Alix Chazottes[(1)] and Houssam Zenati[(3)]

[(1)] Center for Computer Vision (CVN), CentraleSupélec / Opis Team, Inria
[(2)] Laboratory of Signals and Systems (L2S), CentraleSupélec, University Paris-Saclay
[(2)] MIND Team, Inria

{emilie.chouzenoux, alix.chazottes, frederic.pascal}@centralesupelec.fr,
houssam.zenati@inria.fr

`https://pages.saclay.inria.fr/emilie.chouzenoux/ECPml/index.htm`
`http://fredericpascal.blogspot.fr`

Dominante MDS (Mathématiques, Data Sciences)

Sept. - Nov., 2024

CentraleSupélec | université PARIS-SACLAY

Contents

Key references for this course

- Tan, P. N., Steinbach, M., Kumar V., *Data mining cluster analysis: basic concepts and algorithms. Introduction to data mining*. 2013.
- Bishop, C. M. *Pattern Recognition and Machine Learning.* Springer, 2006.
- Hastie, T., Tibshirani, R. and Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction.* Second edition. Springer, 2009.
- James, G., Witten, D., Hastie, T. and Tibshirani, R. *An Introduction to Statistical Learning, with Applications in R.* Springer, 2013

Course 5

(hierarchical) Clustering

Today's Lecture

I. Introduction to Clustering

II. Reminders on Clustering

- Types of methods and clusters
- Distance and Dissimilarity
- Clustering Quality

III. Clustering algorithms

- K-means
- Hierarchical clustering
- DBSCAN
- HDBSCAN

Clustering: An Unsupervised Approach

- Extract homogeneous meaningful or useful categories from the data
- Discover/learn how the data is organized, natural structure
- No ground-truth outputs for training : unsupervised

Objectives

1. Understanding: Biology and medicine, finance, text mining, web, ...
2. Utility: Use cluster characteristics instead of the original data (dimension reduction, regression of high-dimensional data, ...)

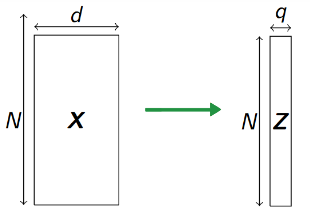
The labels are unknown!

Dimension reduction vs Clustering

Let $\boldsymbol{X} = (x_1, ..., x_N)$ be a set of N training samples

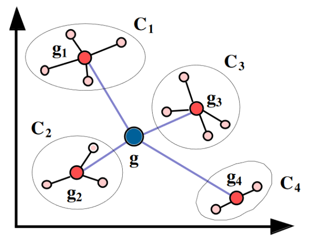
Dimension reduction

- Project $\boldsymbol{X} \in \mathbb{R}^{N,d}$ onto $\boldsymbol{Z} \in \mathbb{R}^{N,q}$ with $q < d$
- Visualize, denoise, reduce computational cost, ...



Clustering

- Groupe similar samples $\boldsymbol{x}_i$ into clusters C_k
- Based on a dissimilarity metric $\mathscr{D}(C_1, C_2)$



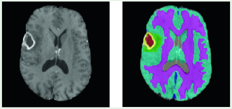
Clustering Applications

Market segmentation

- x: purchase history
- C_k: market segments

Medical image segmentation

- x: image pixels, voxels
- C_k: blood, muscle, tumor, ...



Text mining

- x: text, e-mails, ...
- C_k: folders, themes, ...

Key Questions on Clustering

- Types of clustering ?
- How to characterize a cluster ?
- How to define similarity or dissimilarity between samples ?
- The real/optimal number of clusters ?
- What algorithms can we use and when ?
- How to evaluate a clustering result ? (subjectivity)

Today's Lecture

Today's Lecture

I. Introduction to Clustering

II. Reminders on Clustering

- Types of methods and clusters
- Distance and Dissimilarity
- Clustering Quality

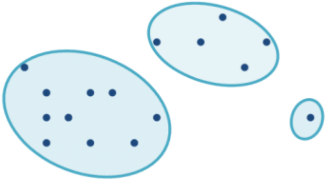
III. Clustering algorithms

- K-means
- Hierarchical clustering
- DBSCAN
- HDBSCAN

Types of clustering: Partitional vs Hierarchical

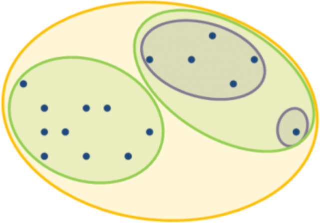
Partitional

- Division into non-overlapping subsets
- Each data point is in exactly one subset



Hierarchical

- Clusters can have sub-clusters
- Set of nested clusters, organized as a tree



Types of Clusters

- Well-separated: Any point in a cluster is closer (or more similar) to every other point in the cluster than to any point not in the cluster.

- Prototype-Based: an object in a cluster is closer (more similar) to the "center" of a cluster, than to the center of any other cluster → Assumptions about shape
  - Center = centroid (average) or medoid (most representative)

- Density-based: dense region of points, which is separated by low-density regions, from other regions of high density. Used when the clusters are irregular or intertwined, and when noise and outliers are present → Is data driven

- Others... graph-based...

Distinctions between sets of clusters

- Exclusive vs non-exclusive (overlapping): separate clusters vs points may belong to more than one cluster
- Fuzzy vs non-fuzzy: each observation $\mathbf{x}_i$ belongs to every cluster $\mathscr{C}_k$ with a given weight $w_k \in [0,1]$ and $\sum_{k=1}^{K} w_k = 1$ (Similar to probabilistic clustering).
- Partial vs Complete: all data are clustered vs there may be non-clustered data, e.g., outliers, noise, "uninteresting background"...
- Homogeneous vs Heterogeneous: Clusters with $\neq$ size, shape, density...

Today's Lecture

I. Introduction to Clustering

II. Reminders on Clustering

- Types of methods and clusters
- Distance and Dissimilarity
- Clustering Quality

III. Clustering algorithms

- K-means
- Hierarchical clustering
- DBSCAN
- HDBSCAN

Dissimilarity Measures

Dissimilarity is a function of the pair (x,y): $\mathscr{D}: \mathbb{E}\times\mathbb{E}\rightarrow\mathbb{R}^+$ s.t

$$\mathscr{D}(x,y)=\mathscr{D}(y,x)\geq 0 \quad \text{and} \quad \mathscr{D}(x,x)=0\ \forall x\in\mathbb{E}$$

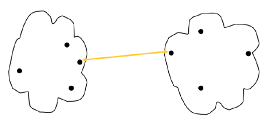
Distance is a dissimilarity measure that satisfies also

1. $\mathscr{D}(x,y)=0 \iff x=y$
2. $\mathscr{D}(x,y)\leq\mathscr{D}(x,z)+\mathscr{D}(z,y)$ (metric)

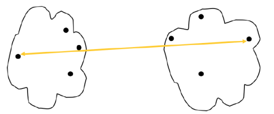
Common distances

- Minkowski: $\mathscr{D}(x,y)=\left(\sum_{j=1}^{d}|x_j-y_j|^q\right)^{\frac{1}{q}}$
  ($q=2\rightarrow$ Euclidian distance, $q=1\rightarrow$: Manhattan distance)
- Mahalanobis: $\mathscr{D}(x,y)=\left[(x-y)^T\Sigma^{-1}(x-y)\right]^{\frac{1}{2}}$
- Hamming: number of indexes where the 2 vectors differ

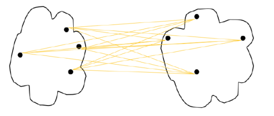
Minimum : $\mathscr{D}(\mathscr{C}_i, \mathscr{C}_j) = \min_{\mathbf{x} \in \mathscr{C}_i, \mathbf{y} \in \mathscr{C}_j} \mathscr{D}(\mathbf{x}, \mathbf{y})$



Maximum : $\mathscr{D}(\mathscr{C}_i, \mathscr{C}_j) = \max_{\mathbf{x} \in \mathscr{C}_i, \mathbf{y} \in \mathscr{C}_j} \mathscr{D}(\mathbf{x}, \mathbf{y})$

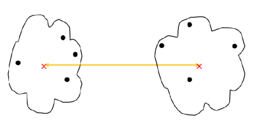


Group Average : $\mathscr{D}(\mathscr{C}_i, \mathscr{C}_j) = \dfrac{1}{n_i\, n_j} \sum_{\mathbf{x} \in \mathscr{C}_i} \sum_{\mathbf{y} \in \mathscr{C}_j} \mathscr{D}(\mathbf{x}, \mathbf{y})$



Between Centroids : $\mathscr{D}(\mathscr{C}_i, \mathscr{C}_j) = \mathscr{D}(m_i, m_j)$,

with $m_i = \dfrac{1}{n_i} \sum_{\mathbf{x} \in \mathscr{C}_i} \mathbf{x}$



Dissimilarity Between Clusters (2/2)

Objective function distances

- Ward distance: $\mathscr{D}(\mathscr{C}_i, \mathscr{C}_j) = \sqrt{\dfrac{2\, n_i\, n_j}{n_i + n_j}}\mathscr{D}(m_i, m_j)$
- WPGMA (Weighted Pair Group Method with Arithmetic Mean) recursive distance

$$\mathscr{D}(\mathscr{C}_i, \mathscr{C}_j) == \frac{\mathscr{D}(\mathscr{C}_i^1, \mathscr{C}_j) + \mathscr{D}(\mathscr{C}_i^2, \mathscr{C}_j)}{2}$$

where $\mathscr{C}_i^1, \mathscr{C}_i^2$ are the child clusters of $\mathscr{C}_i$

Today's Lecture

I. Introduction to Clustering

II. Reminders on Clustering

- Types of methods and clusters
- Distance and Dissimilarity
- Clustering Quality

III. Clustering algorithms

- K-means
- Hierarchical clustering
- DBSCAN
- HDBSCAN

What makes a good clustering ?

- Centroid: $m_i = \dfrac{1}{n_i} \sum_{\mathbf{x} \in \mathscr{C}_i} \mathbf{x}$
- Inertia: $J_i = \sum_{\mathbf{x} \in \mathscr{C}_i} \mathscr{D}^2(\mathbf{x}, m_i)$

  (low J_i corresponds to a smaller dispersion of points around m_i.)
- Within distance: $J_w = \sum_i \sum_{\mathbf{x} \in \mathscr{C}_i} \mathscr{D}^2(\mathbf{x}, m_i) = \sum_i J_i$
- Between distance: $J_b = \sum_i n_i \mathscr{D}^2(m_i, m)$

  where m is the sample mean $m = \dfrac{1}{n} \sum \mathbf{x}$
- Performance measures: accuracy (when ground truth is known), ARI (Adjusted Rand Index), AMI (Adjusted Mutual Information)...

A good clustering...

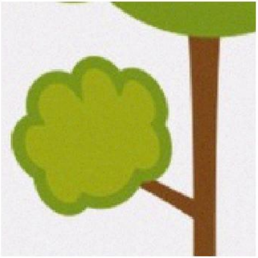
Minimizes the within distance J_w and maximizes the between distance J_b

Illustrative example

Objective

Cluster noisy data for a segmentation application in image processing

(a) Tree data



(b) Noisy tree data

Figure: Data on which the clustering algorithms are evaluated

Should be easy...

Today's Lecture

Today's Lecture

K-means

It is a prototype-based clustering technique.

Notations: n unlabelled data vectors of $\mathbb{R}^p$ denoted as $\mathbf{x} = (\mathbf{x}_1, \ldots, \mathbf{x}_n)$ which should be split into K classes $\mathscr{C}_1, \ldots, \mathscr{C}_K$, with $\mathrm{Card}(\mathscr{C}_k) = n_k, \sum_{k=1}^{K} n_k = n$.

Centroid of $\mathscr{C}_k$ is denoted m_k.

Optimal solution

Number of partitions of $\mathbf{x}$ into K subsets:

$$P(n, K) = \frac{1}{K!} \sum_{k=0}^{K} k^n \, (-1)^{K-k} \, C_K^k \text{ for } K < n$$

where $C_K^k = \dfrac{K!}{k!\,(K-k)!}$.

Example: $P(100, 5) \approx 10^{68}$!!!!

K-means algorithm

- Partitional clustering approach where K of clusters must be specified
- Each observation is assigned to the cluster with the closest centroid
- Minimizes the intra-cluster variance $V = \sum_k \sum_{i|\mathbf{x}_i \in \mathscr{C}_k} \frac{1}{n_k} ||\mathbf{x}_i - m_k||^2$
- The basic algorithm is very simple

Algorithm 1 K-means algorithm

Input : $\mathbf{x}$ observation vectors and the number K of clusters
Output : $\mathbf{z} = (z_1, \ldots, z_N)$, the labels of $(\mathbf{x}_1, \ldots, \mathbf{x}_N)$
Initialization : Randomly select K points as the initial centroids
Until convergence (define a criterion, e.g. error, changes, centroids estimation...) **Repeat**

1. Form K clusters by assigning $\mathbf{x}_i$ to the closest centroid m_k
   $C_k = \{\mathbf{x}_i,\ \forall i \in \{1,...,n\} \mid\ d(\mathbf{x}_i, m_k) \leq d(\mathbf{x}_i - m_j)\ , \forall j \in \{1,...,K\}\ \}$
2. Recompute the centroids
   $\forall k \in \{1,...,K\} : m_k = \frac{1}{n_k} \sum_{\mathbf{x}_i \in \mathscr{C}_k} \mathbf{x}_i.$

K-means drawbacks and alternatives

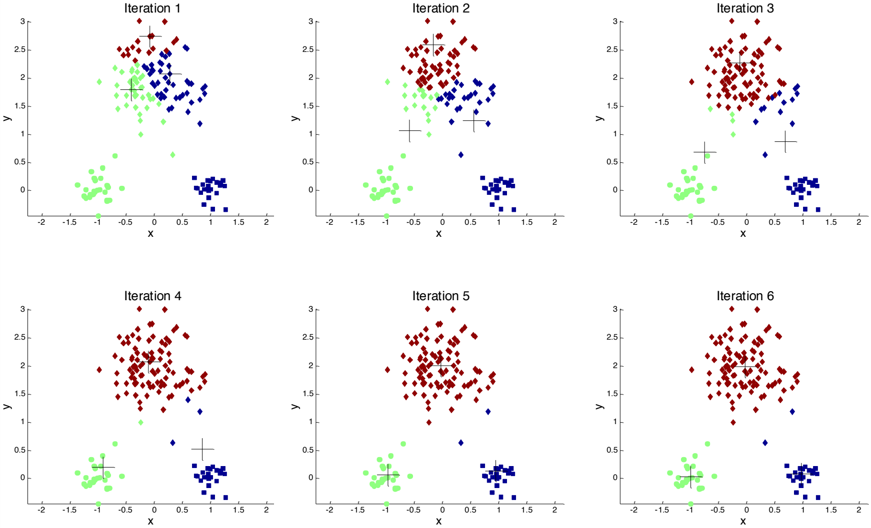
K-means is simple but ...

- Solution depends on initialization
- Need to know K in advance
- Can't handle noise or outliers : *non-robust*
- Fails with clusters of *non-convex* shapes

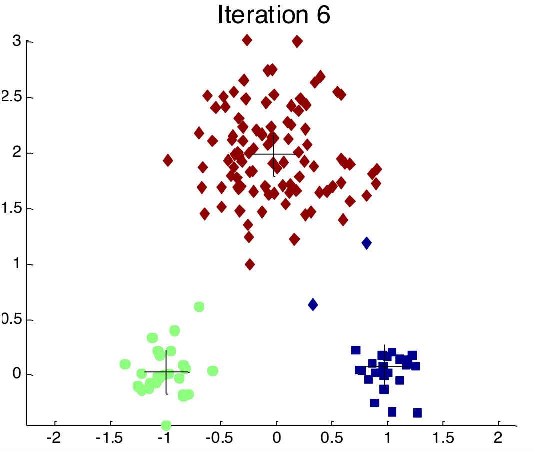
Several alternatives

- K-means++: Seeding algorithm to initialize clusters with centroids "spread-out" throughout the data
- K-medoids: To address the robustness aspects
- Kernel K-means: For overcoming the convex shape
- Many others ...

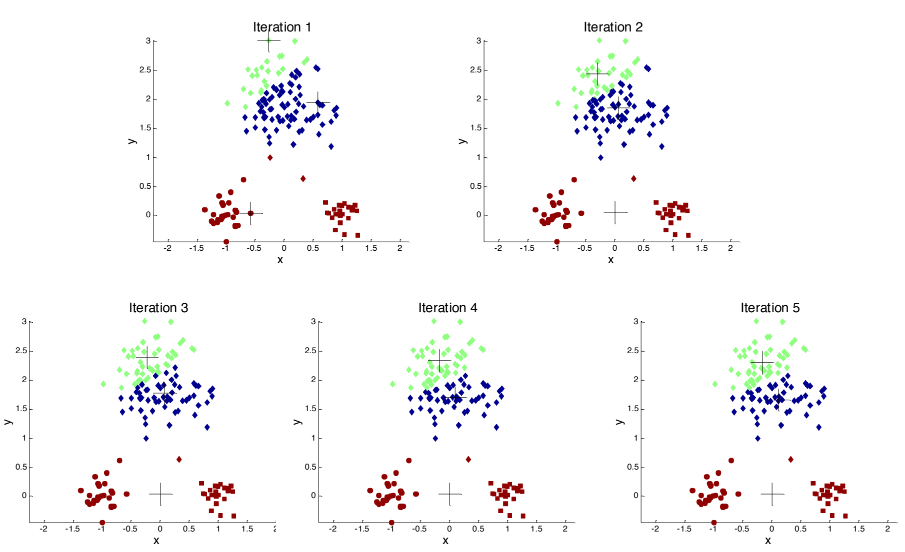
Correct initilization



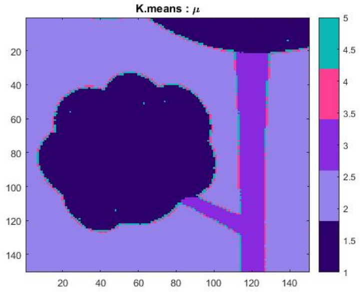
Correct initilization



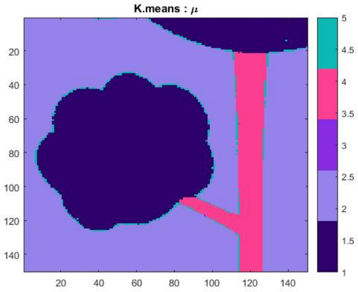
Bad initialization



Results on the data set



(a) K-means++



(b) "Clusters"

Figure: Clustering obtained with two different initialization techniques

Comments...

Today's Lecture

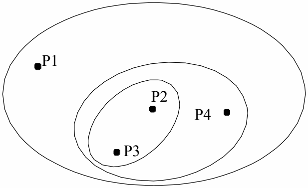
Hierarchical clustering Principles

- Produces a set of nested clusters organized as a hierarchical tree → bypass choice of K
- Can be visualized as a dendrogram: a tree like diagram that records the sequences of merges or splits with branch length corresponding to cluster distance

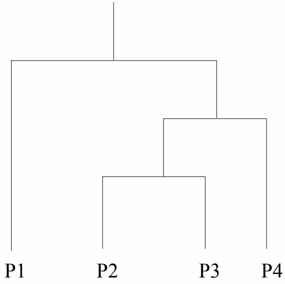
Two approaches

1. Agglomerative: *Bottom-up* - Start with as much clusters as observations and iteratively *aggregate* observations using a given *distance*
2. Divise: *Top-down* - Start with one cluster containing all observations and iteratively *split* into smaller clusters

Hierarchical Clustering: The tree



(a) Hierarchical Clusters

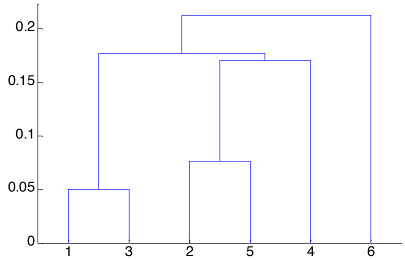


(b) Dendrogram

We can see that ...

- Each node (cluster) in the tree (except the leaf nodes) is the union of its children (subclusters)
- The root of the tree is the cluster containing all objects.

Hierarchical clustering example



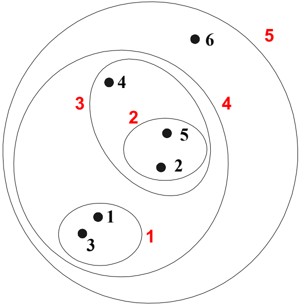


Figure: General principles

Inter-Cluster distance

Most popular clustering techniques

Algorithm 2 Agglomerative hierarchical clustering

Input : $\mathbf{x}$ observation vectors and "cutting" threshold λ
Output : all merged clusters set (at each iteration) and "inter-cluster" distances (between clusters)
Initialization : $n =$ sample size $=$ number of clusters.

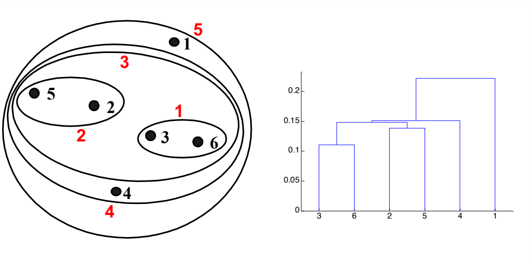
While Number of clusters > 1

1. Compute distances between clusters
2. Merged the two nearest clusters

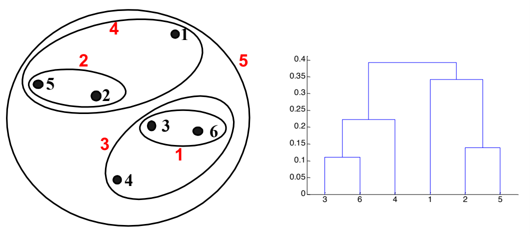
Inter-Cluster distances

- MIN → Single Linkage: $d(\mathscr{C}_i, \mathscr{C}_j) = \min_{\mathbf{x}\in\mathscr{C}_i, \mathbf{y}\in\mathscr{C}_j} d(\mathbf{x}, \mathbf{y})$
- MAX → Complete Linkage: $d(\mathscr{C}_i, \mathscr{C}_j) = \max_{\mathbf{x}\in\mathscr{C}_i, \mathbf{y}\in\mathscr{C}_j} d(\mathbf{x}, \mathbf{y})$
- Group Average → Average Linkage: $d(\mathscr{C}_i, \mathscr{C}_j) = \dfrac{1}{n_i\, n_j} \sum_{\mathbf{x}\in\mathscr{C}_i} \sum_{\mathbf{y}\in\mathscr{C}_j} d(\mathbf{x}, \mathbf{y})$
- Between centroid → Centroid Linkage: $d(\mathscr{C}_i, \mathscr{C}_j) = d(m_i, m_j)$, with $m_i = \dfrac{1}{n_i} \sum_{\mathbf{x}\in\mathscr{C}_i} \mathbf{x}$
- Objective function → Objective Linkage:
  - Ward distance $d(\mathscr{C}_i, \mathscr{C}_j) = \sqrt{\dfrac{2\, n_i\, n_j}{n_i + n_j}} d(m_i, m_j)$
  - WPGMA (Weighted Pair Group Method with Arithmetic Mean) recursive distance $d(\mathscr{C}_i, \mathscr{C}_j) == \dfrac{d(\mathscr{C}_i^1, \mathscr{C}_j) + d(\mathscr{C}_i^2, \mathscr{C}_j)}{2}$ where $\mathscr{C}_i^1, \mathscr{C}_i^2$ are the child clusters of $\mathscr{C}_i$
  - ...

Different distances ⇒ different results



(a) MIN



(b) MAX

Different distances ⇒ different results

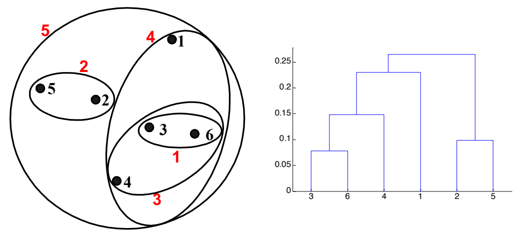
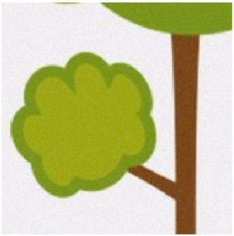


Figure: Group average

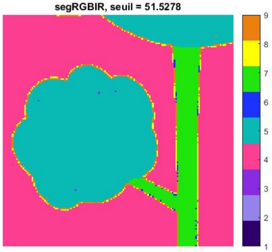
Ward: very similar results.

- MIN : can handle non-elliptical shape BUT sensitive to outliers, noise...
- MAX: less sensitive to outliers BUT can break large clusters and biased towards globular clusters
- Average: don't break large clusters BUT biased towards globular clusters
- Ward: Hierarchical analogue of K-means

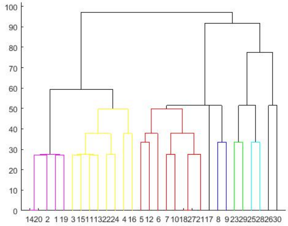
Results on the data set - Single Linkage



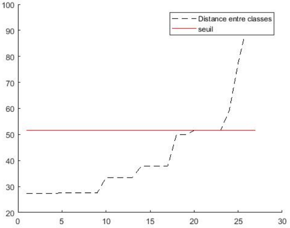
(a) Noisy Tree



(b) Single Linkage

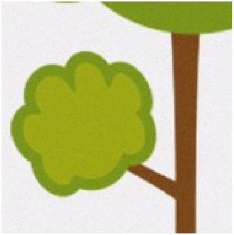


(c) Dendrogram

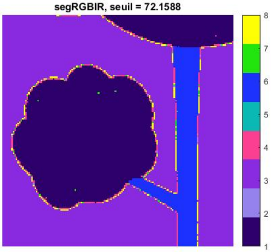


(d) Cutting Threshold

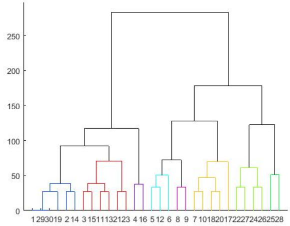
Results on the data set - Complete Linkage



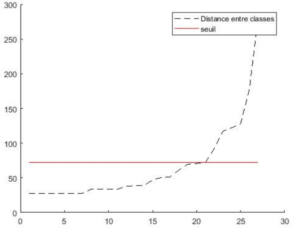
(e) Noisy Tree



(f) Complete Linkage

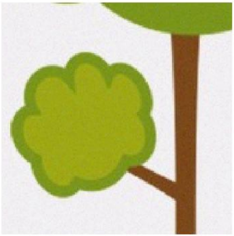


(g) Dendrogram

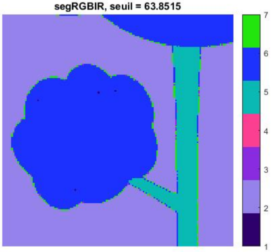


(h) Cutting Threshold

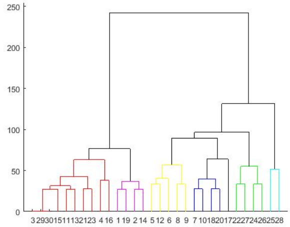
Results on the data set - Average Linkage



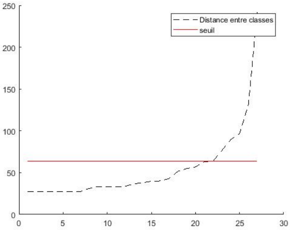
(i) Noisy Tree



(j) Average Linkage

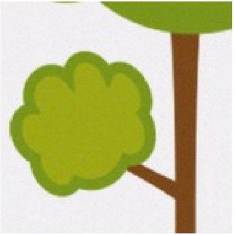


(k) Dendrogram

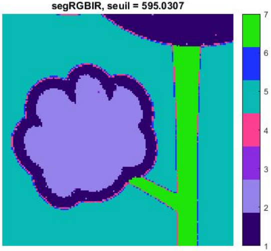


(l) Cutting Threshold

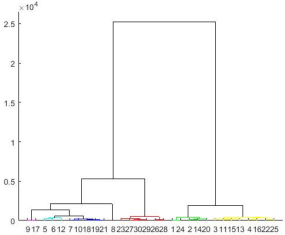
Results on the data set - Ward Linkage



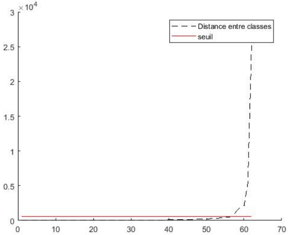
(m) Noisy Tree



(n) Average Linkage

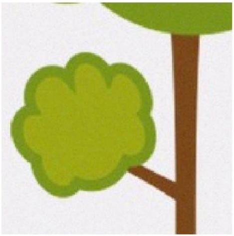


(o) Dendrogram

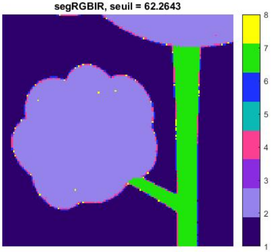


(p) Cutting Threshold

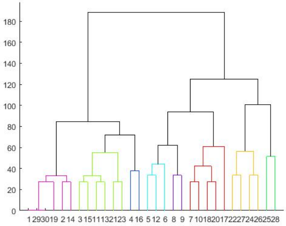
Results on the data set - WPGMA Linkage



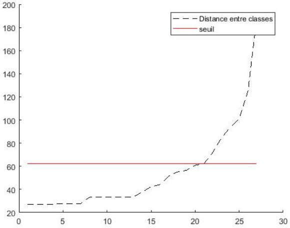
(q) Noisy Tree



(r) Average Linkage



(s) Dendrogram



(t) Cutting Threshold

Hierarchical clustering - Pros and cons

- Pros
  - Simple and intuitive
  - Unsupervised: no *a priori* assumptions
  - Interpretable: number of clusters, used distance...
- Cons
  - Computational cost: single linkage (O(n^3),O(n^2) or O(n)), complete linkage (O(n^3) or O(n^2)), average (O(n^3)), Ward's method (O(n^3)), ...
  - Cutting threshold: challenging choice!
  - Lack of robustness: sensitivity to outliers and noise
  - No global objective function to optimize
  - Handle heterogeneous data (clusters of $\neq$ size, non-globular shapes...)

Today's Lecture

DBSCAN : A Density-based Algorithm

For an observation $\mathbf{x}_i$, find a sufficiently (MinPts) large neighborhood (ε), then

- aggregate the new observations (neighbors) to the cluster $\mathscr{C}_k$ of $\mathbf{x}_i$,
- else $\mathbf{x}_i$ is an isolated observation (outlier).

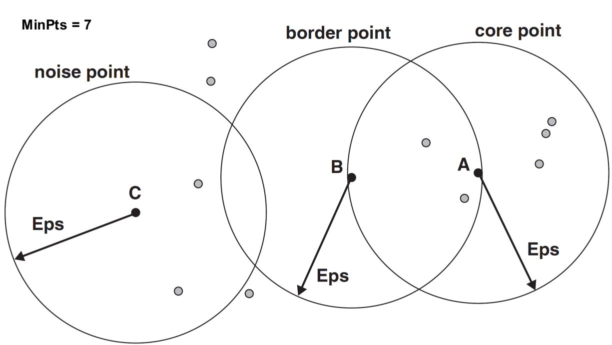
This results in three types of points called core, border, or noise points.

Key parameters

- ε and ε-neighborhood: $\mathcal{N}_\varepsilon(\mathbf{x}_i) = \{\mathbf{z} | d(\mathbf{x}_i, \mathbf{z}) < \varepsilon\}$
- MinPts: n_{min} for defining core points $\mathbf{x}_i$ s.t. $\operatorname{card}(\mathcal{N}_\varepsilon(\mathbf{x}_i)) \geq n_{min}$

DBSCAN: Three Types of Points

1. Core point: is near the center of a cluster/has MinPts neighbors
2. Border point: is not a core point, but is in the neighborhood of a core point
3. Noise point: is any point that is neither a core nor a border point

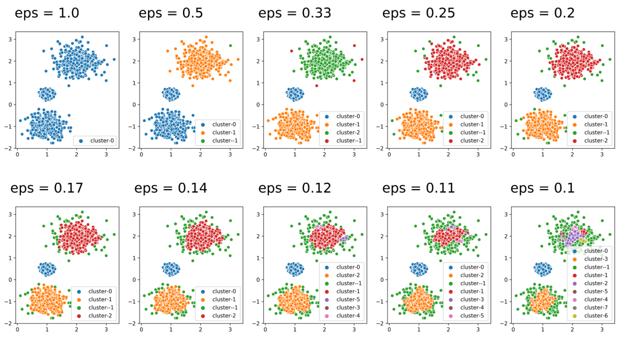


DBSCAN: Influence of ϵ

The parameter ϵ represents the minimum distance between two non-neighboring points:

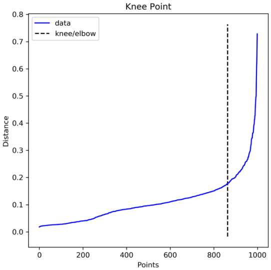
→ A very large ϵ causes all possible clusters to merge into one cluster

→ A very small ϵ leads to a lot of noise, points are not assigned to clusters



DBSCAN: So how do we choose ϵ ?

- Depends on the distance between the data points
- The Elbow trick on the k-NN plot is commonly used in practice (k is MinPts!):
  - → x-axis all the points
  - → y-axis the average distance of each point to their its k-NN
- Remains a difficult choice!



DBSCAN algorithm

Algorithm 3 DBSCAN algorithm

Input: $\mathbf{x}$ observations, ε, MinPts

Output: $\mathcal{Z}$, labels of $\mathbf{x}$

For all $\mathbf{x}_i$

1. Verify that $\mathbf{x}_i$ has not been visited by the algo, else $\mathbf{x}_i$ is marked "as visited"
2. Identify the ε-neighborhood of $\mathbf{x}_i$, $\mathcal{N}_\varepsilon(\mathbf{x}_i)$.
3. **If** $\text{card}(\mathcal{N}_\varepsilon(\mathbf{x}_i)) \leq n_{min}$, then mark P as an isolated point.
   Else Create a cluster $\mathscr{C}_k$ containing $\mathbf{x}_i$ and run class_extension($\mathscr{C}_k, \mathbf{x}_i, \varepsilon, n_{min}$)

Cluster extension

Algorithm 4 Extension class function

Input: Cluster $\mathscr{C}_k$ to increase, observation $\mathbf{x}_i$ of $\mathscr{C}_k$, n_{min}, ε.

Output : $\mathscr{Z}$ labels of observations in $\mathscr{N}_\varepsilon(\mathbf{x}_i)$

Forall $\mathbf{x}_j, i \neq j$ of $\mathscr{N}_\varepsilon(\mathbf{x}_i)$

1. Verify that $\mathbf{x}_j$ has not been visited by the algo, else $\mathbf{x}_i$ is marked "as visited"
2. Identify the ε-neighborhood of $\mathbf{x}_j$, $\mathscr{N}_\varepsilon(\mathbf{x}_j)$.
3. **If** $\text{card}(\mathscr{N}_\varepsilon(\mathbf{x}_j)) \geq n_{min}$
   $\mathscr{N}_\varepsilon(\mathbf{x}_i) = \mathscr{N}_\varepsilon(\mathbf{x}_i) + \mathscr{N}_\varepsilon(\mathbf{x}_j)$
4. **If** $\mathbf{x}_j$ is not clustered, add to $\mathscr{C}_k$.

Illustration of DBSCAN principles

See the following video : [here]()
Several examples can be found [here]()

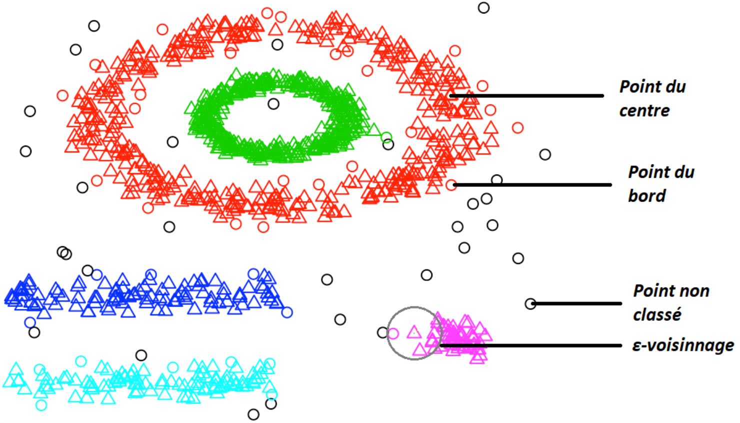
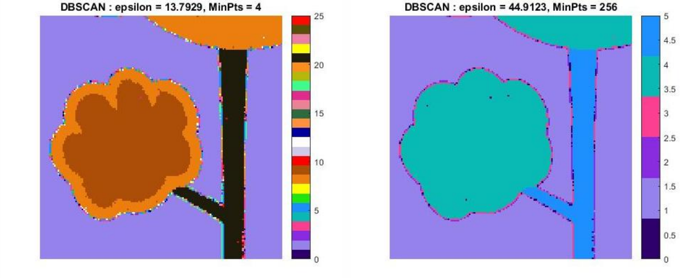


Figure: Clustering results obtained with DBSCAN algorithm.

Results on the data set - DBSCAN



(a) MinPts = 4 (b) MinPts = 256

Figure: Influence of MinPts and ε

Discussion: ε, number of clusters, MinPts...

- Pros: Resistant to Noise, can handle clusters of different shapes and sizes
- Cons: Interpretable parameters (estimation), Varying densities, High-dimensional data

Algorithms comparison

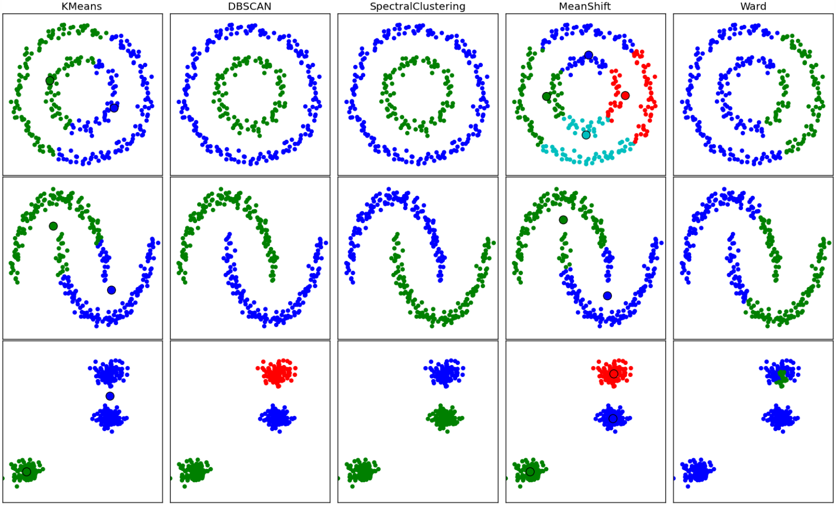


Figure: From Scikits learn: `https://ogrisel.github.io/scikit-learn.org/sklearn-tutorial/modules/clustering.html`

Today's Lecture

HDBSCAN

Key Idea: Convert DBSCAN into a hierarchical clustering algorithm and

→ bypass the choice of the ϵ-parameter!

→ scan all possible solutions with all values of ϵ

Main steps:

1. Transform the space according to the density/sparsity
2. Build the minimum spanning tree of the distance weighted graph
3. Construct a cluster hierarchy of connected components.
4. Condense the cluster hierarchy based on minimum cluster size.
5. Extract the stable clusters from the condensed tree.

Easier to understand with an example!

Campello, R.J., Moulavi, D. and Sander, J., *"Density-based clustering based on hierarchical density estimates"*. In Pacific-Asia conference on knowledge discovery and data mining (pp. 160-172). Springer, Berlin, Heidelberg, April 2013.

HDBSCAN

Key Idea: Convert DBSCAN into a hierarchical clustering algorithm and

→ bypass the choice of the ϵ-parameter!

→ scan all possible solutions with all values of ϵ

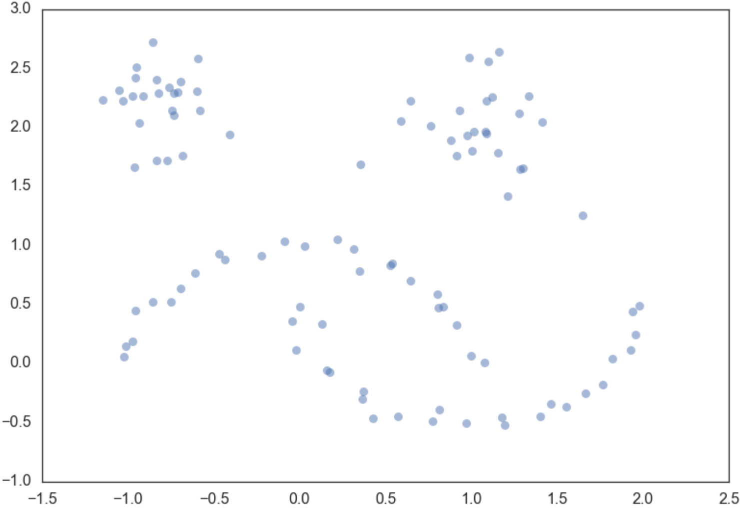
Main steps:

1. Transform the space according to the density/sparsity
2. Build the minimum spanning tree of the distance weighted graph
3. Construct a cluster hierarchy of connected components.
4. Condense the cluster hierarchy based on minimum cluster size.
5. Extract the stable clusters from the condensed tree.

Easier to understand with an example!

Campello, R.J., Moulavi, D. and Sander, J., *"Density-based clustering based on hierarchical density estimates"*. In Pacific-Asia conference on knowledge discovery and data mining (pp. 160-172). Springer, Berlin, Heidelberg, April 2013.

HDBSCAN: Illustrative example



©https://hdbscan.readthedocs.io/en/latest/how_hdbscan_works.html

Step 1: Transform The Space

- Goal: Prepare the data for a single linkage clustering (real data is noisy and single linkage is not robust!)
- Key idea: Push sparse points away from the rest of the data before clustering
- The *islands/sea* analogy → Make *sea* points more distant from each other and from the *land*

How do we evaluate density ?

- Need an inexpensive density estimate ⇒ k-NN is the simplest
- Call it the core distance for parameters k and point $\mathbf{x}_i$, $\mathrm{core}_k(\mathbf{x}_i)$

And how do we connect points now ?

Step 1: Transform The Space

- Goal: Prepare the data for a single linkage clustering (real data is noisy and single linkage is not robust!)
- Key idea: Push sparse points away from the rest of the data before clustering
- The *islands/sea* analogy → Make *sea* points more distant from each other and from the *land*

How do we evaluate density ?

- Need an inexpensive density estimate ⇒ k-NN is the simplest
- Call it the core distance for parameters k and point $\mathbf{x}_i$, $\mathrm{core}_k(\mathbf{x}_i)$

And how do we connect points now ?

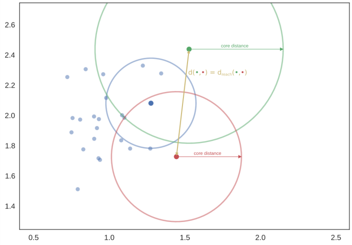
Step 1 : Mutual Reachability Distance

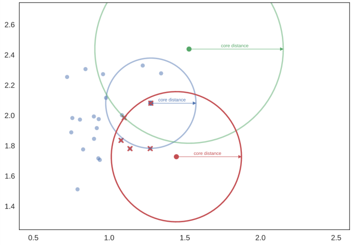
A new distance metric is defined as

$$d_{mreach-k}(\mathbf{x}_i, \mathbf{x}_j) = \max(\text{core}_k(\mathbf{x}_i), \text{core}_k(\mathbf{x}_j), d(\mathbf{x}_i, \mathbf{x}_j)),$$

Meaning that we want to connect points that are

1. Close enough to each other : $d(\mathbf{x}_i, \mathbf{x}_j)$
2. In a dense enough region : $\text{core}_k(\mathbf{x}_i)$



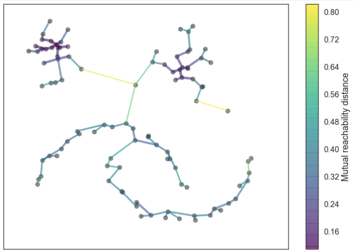


Step 2 : The Minimum Spanning Tree

- Goal: Prepare the data for clustering using d_{mreach}
- Key ideas:
  - Construct a graph that connects all points
  - Start disconnecting them by lowering a threshold (sea level drops)
  - Points are the vertices and the *edges* are weighted by d_{mreach}
  - n^2 possible edges → the minimum spanning tree

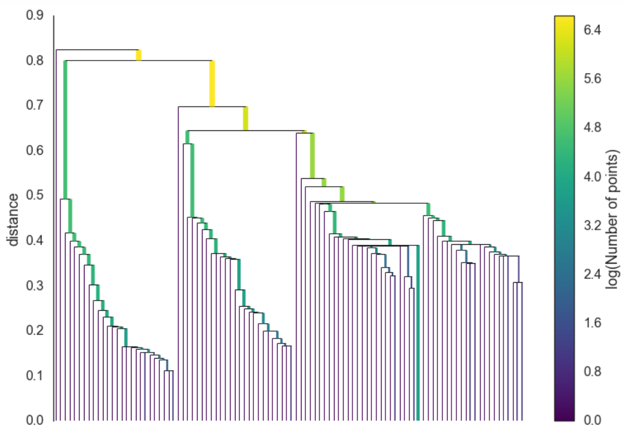
Algorithms from graph theory

- Prim's algorithm
- Dual Tree Boruvka



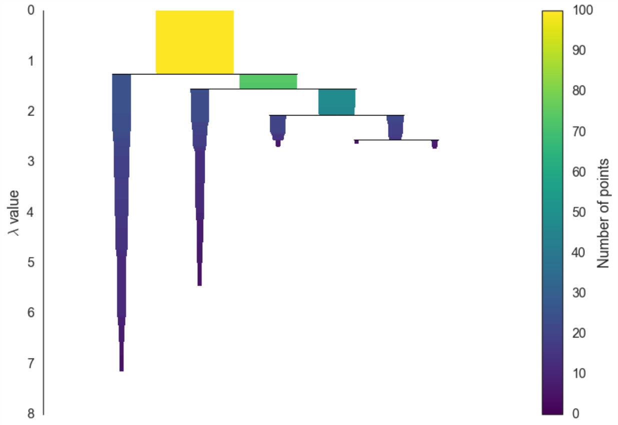
Step 3: Build the cluster hierarchy

Clusters emerge progressively as we lower the d_{mreach} threshold ($\rightarrow$ sort the edges and start single linkage)



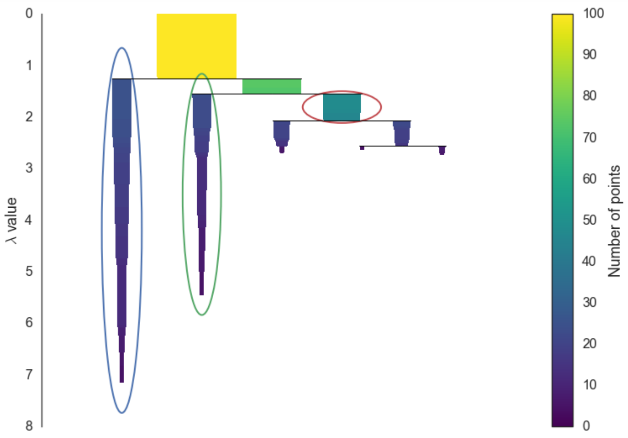
Step 4 : Condense the cluster tree

Get rid of levels that resulted in noise : nbr of points $\leq C_{\min}$
(clusters are shrinking $\neq$ splitting)

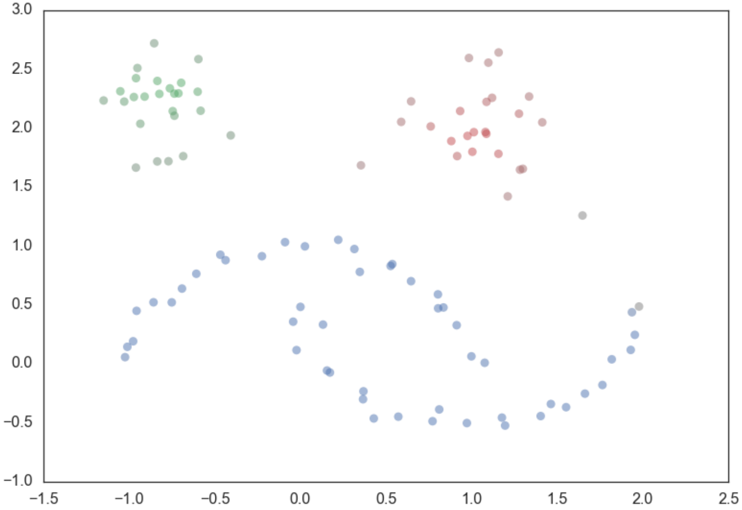


Extract the clusters

Key idea: Choose clusters that persist (live for a long time) and that are large → maximize a stability criterion
(flat clustering: can't select descendance of a selected cluster!)



Results



HDBSCAN: Summary

Implementation: The 5 main steps

1. Compute $\mathrm{core}_k(\mathbf{x}_i)$ using $MinPts$ → Measure density
2. Transform the space: use new metric d_{mreach}

   → Robustness to noise!
3. Construct a minimum spanning tree

   → Lower computational cost
4. Simplify/condense the tree using $C_{\min}$

   → Preprocessing for the next step
5. Extract final clustering results

   → Maximize cluster stability

In conclusion: Two parameters ($MinPts$ and $C_{\min}$), varying densities, robust to outliers, interpretability...

HDBSCAN: Summary

Implementation: The 5 main steps

1. Compute $\mathrm{core}_k(\mathbf{x}_i)$ using $MinPts$ → Measure density
2. Transform the space: use new metric d_{mreach}
   → Robustness to noise!
3. Construct a minimum spanning tree
   → Lower computational cost
4. Simplify/condense the tree using $C_{\min}$
   → Preprocessing for the next step
5. Extract final clustering results
   → Maximize cluster stability

In conclusion: Two parameters ($MinPts$ and $C_{\min}$), varying densities, robust to outliers, interpretability...

HDBSCAN: Summary

Implementation: The 5 main steps

1. Compute $\mathrm{core}_k(\mathbf{x}_i)$ using $MinPts$ → Measure density
2. Transform the space: use new metric d_{mreach}

   → Robustness to noise!
3. Construct a minimum spanning tree

   → Lower computational cost
4. Simplify/condense the tree using C_{min}

   → Preprocessing for the next step
5. Extract final clustering results

   → Maximize cluster stability

In conclusion: Two parameters ($MinPts$ and C_{min}), varying densities, robust to outliers, interpretability...

HDBSCAN: Summary

Implementation: The 5 main steps

1. Compute $\mathrm{core}_k(\mathbf{x}_i)$ using $MinPts$ → Measure density
2. Transform the space: use new metric d_{mreach}

   → Robustness to noise!
3. Construct a minimum spanning tree

   → Lower computational cost
4. Simplify/condense the tree using $C_{\min}$

   → Preprocessing for the next step
5. Extract final clustering results

   → Maximize cluster stability

In conclusion: Two parameters ($MinPts$ and $C_{\min}$), varying densities, robust to outliers, interpretability...

HDBSCAN: Summary

Implementation: The 5 main steps

1. Compute $\mathrm{core}_k(\mathbf{x}_i)$ using $MinPts$ → Measure density
2. Transform the space: use new metric d_{mreach}

   → Robustness to noise!
3. Construct a minimum spanning tree

   → Lower computational cost
4. Simplify/condense the tree using C_{min}

   → Preprocessing for the next step
5. Extract final clustering results

   → Maximize cluster stability

In conclusion: Two parameters ($MinPts$ and C_{min}), varying densities, robust to outliers, interpretability...